

# PrefixCDD: Effective Online Concept Drift Detection over Event Streams using Prefix Trees

Jesús Huete  
Eindhoven University of Technology  
Eindhoven, The Netherlands  
jesushuete92@gmail.com

Abdulhakim A. Qahtan  
Utrecht University  
Utrecht, The Netherlands  
a.a.a.qahtan@uu.nl

Marwan Hassani  
Eindhoven University of Technology  
Eindhoven, The Netherlands  
m.hassani@tue.nl

**Abstract**—Process mining focuses on applying data mining techniques over business process data. Recently, with the improvements in sensing, collection, and storage of event data, a big demand for both shorter mining time and adaptive models of streaming process events arose. This increased the interest in streaming process mining. Some techniques within this field attempt to identify drifts (change points) from evolving process data streams. Existing work on supervised and unsupervised-learning approaches over data streams have several limitations with regards to the nature of the drifts, the excessive storage required to store and process the stream, and the performance over real-world datasets. This paper contributes *PrefixCDD*, an efficient unsupervised-learning novel approach for online concept drift detection (CDD) over event streams. Our proposed approach utilizes a data structure, where the data stream components are stored in a set of prefix-trees. It transforms then the discrete data into continuous one using a Principal Component Analysis (PCA) approach over the trees. Then, ADWIN is used to focus on up-to-date information, making it appealing to work with the decaying mechanism logic behind our algorithm. Using six artificial and three real-life datasets, *PrefixCDD* outperforms state-of-the-art techniques in terms of detecting existing drifts of different natures, discovering them shortly after they appear, and the overall execution time.

**Index Terms**—Process Mining, Stream Process Mining, Concept Drift Detection, Unsupervised-Learning, Event Streams.

## I. INTRODUCTION

Business processes are continuously changing in response to external factors, like governmental regulations, variations in demand or supply, or seasonal factors. Process mining became in the meanwhile an established field which takes methods and techniques from data mining and applies them over logs of process information. These logs contain information regarding the activities performed by the business process, connected by timestamps of execution, and related to a specific case. Process mining tasks involve the extracting knowledge from the logs to discover a process model [1] [2], monitor performance of these models with conformance checks [3] [4], or improve existing models with process optimization techniques [5].

One important application in the field of process mining, which has been recently introduced, is related to change detection. As stated in [6], processes can change with respect to three main process perspectives: control-flow, data, and resource. Such changes introduce a drift in the concept (process behavior). This for example can be related to the way in which

activities are executed (when, how, and by whom). In the context of this paper, we are focusing on changes from the control-flow perspective. The basic premise in handling concept drifts is that *the characteristics of the traces before the change point differ from the characteristics of the traces after the change point*. The problem is then to identify the points in time when the process has changed (if any). Bose et al. [6] argue that it involves two primary steps: 1) capturing the characteristics of the traces, and 2) identifying when these characteristics change. There are four different classes of drifts: *Sudden drift* correspond to a substitution of an existing process with a new process. In *gradual drift* processes coexist for some time, where the first one is gradually discontinued. In *Recurring drift* a set of processes reappear after some time (substituted back and forth). *Incremental drift* refer to the substitution between two processes done via smaller incremental changes. The method presented in this paper focuses on detecting sudden drifts. However, our framework was able to additionally detect gradual drifts in a one of our experimental evaluation datasets (cf. the results on BPI 2020 in Section V-E).

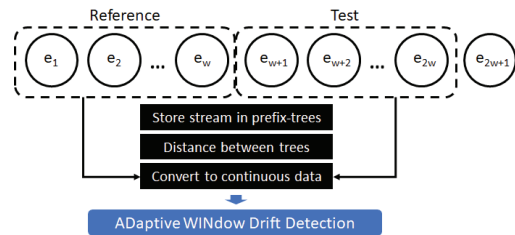


Fig. 1. Overview of the proposed *PrefixCDD* framework.

Current unsupervised-learning CDD techniques show the following limitations: (i) imply some a-priori knowledge of the possible nature of the drift, (ii) dismiss important information from the traces, (iii) perform poorly over real-life event logs, and (iv) require process discovery techniques before identifying drifts. Therefore, an unsupervised-learning online drift detection technique, that is capable of addressing the aforementioned limitations, is required. We propose *PrefixCDD* (Fig. 1) in this context, with the following contributions:

- 1) A novel framework for online detection of concept drifts over event streams,

- 2) Using a unique combination of an efficient data structure, an innovative PCA-based distance measure, and a well-established adaptive drift detection technique,
- 3) An accurate algorithm in terms of quality of detection and time of execution, proven extensively over six synthetic and three real-world datasets, and
- 4) Detecting different types of sudden and gradual drifts.

The remainder of this paper is structured as follows: Section II shows an overview of the related work. Section III introduces required notation and then the framework is defined in Section IV. Section V elaborates on the experimental evaluation before concluding the paper in Section VI.

## II. RELATED WORK

This section covers the most related literature to the proposed framework from two relevant domains. There are two lines of research that are related to our work.

**Stream Process Mining:** Advances in information technology lead to the generation of large flows of data, which can be mined for interesting and relevant information in a variety of applications. The volume of this data has lead to a number of computational and mining challenges according to [7]: with increasing volume of data, it is no longer possible to process the data efficiently by using multiple passes. Rather, one can process a data item at most once. This leads to constraints on the implementation of the underlying algorithms. An event stream then would be a specific type of data stream, where each data point refers to an event generated by an underlying process or system.

**Concept Drift Detection:** Recent work attempts to apply concept drift detection techniques into data streams. The problem with supervised learning approaches is that they assume that the true labels are readily available after the prediction. In reality, this is rarely the case. In most real-world scenarios where a process generates these labels, they can take too long to be available, if ever. Because of these restrictions for supervised-learning techniques over event streams, this paper focuses on identifying unsupervised-learning CDD techniques. Unsupervised window-based change detection, according to [8], is based on comparing the distribution in a current stream window with a reference distribution, where density estimation techniques, divergence metrics, and distance functions are essential to model and compare the distributions. Modeling the data distribution becomes a challenge when working with multidimensional data. Also, as data dimensionality increases, so does the computational costs and inaccuracy of the techniques. Table I shows a summary of the unsupervised-learning techniques reviewed for this paper. The comparison factors help us identify which of these techniques work in a similar fashion to our proposed framework.

The Robust and Accurate Approach [14] is a G-test of independence over frequencies of the directly-follows relations from the log. The approach from [10] identifies deviating traces based on their LOF score. [11] performs a statistical G-test of independence is performed over distributions of special behavioral relations between events, as observed in two

| Algorithm               | Disc→Cont            |                    | Window            |         | Drift Type |        |         |             |
|-------------------------|----------------------|--------------------|-------------------|---------|------------|--------|---------|-------------|
|                         | Probability Function | Feature Extraction | Distance Function | Dynamic | Fixed      | Sudden | Gradual | Incremental |
| StrProMCDD [9]          |                      |                    | ✓                 | ✓       |            | ✓      |         |             |
| PCA-CD [8]              | ✓                    |                    |                   | ✓       |            | ✓      |         |             |
| *DOA [10]               |                      |                    | ✓                 | ✓       |            |        |         | ✓           |
| UDD [11]                |                      |                    | ✓                 | ✓       |            | ✓      |         |             |
| ProDrift [12]           |                      | ✓                  |                   | ✓       |            | ✓      | ✓       |             |
| TC [13]                 |                      | ✓                  |                   |         | ✓          | ✓      |         |             |
| *Robust [14]            |                      | ✓                  |                   | ✓       |            | ✓      | ✓       |             |
| <b>PrefixCDD (this)</b> |                      |                    | ✓                 | ✓       |            | ✓      | ✓       |             |

TABLE I  
UNSUPERVISED CONCEPT DRIFT DETECTION TECHNIQUES  
CHARACTERISTICS (\* = OFFLINE APPROACHES).

adjacent windows of adaptive size. Similarly, [12] performs a Chi-squared test of independence over a contingency matrix of *runs* frequencies. [8] applies Principal Component Analysis (PCA) to project the original multidimensional data stream into a lower dimensional space and performs a change-score over defined Principal Components (PCs).

## III. PRELIMINARIES

In this paper, we use  $S = \langle e_1, e_2, e_3, \dots \rangle$  to represent event streams. Note that, an event stream is a potentially infinite sequence of events, where events are ordered and indexed by time. We define a trace as a sequence of events correlated to a case and ordered by time. Furthermore, events of the same trace do not need to be consecutive in the event stream, i.e. traces can "overlap".

We represent each event as  $e = (c, a, t)$ , where  $c$  represents the case identifier,  $a$  refers to the activity label, and  $t$  to the time-stamp. Each event is unique, as the combination of the three attribute values cannot be repeated. Yet, multiple events may share the same activity label. Though most stream generation tools provide a unique identifier for each event, such information is not needed for the purpose of this framework.

These events arriving through the event stream will be stored in a set of prefix-trees. The trees, which will be covered in Section IV are composed of nodes. Each node stores a set of attributes which are represented as  $n = (i, a, p, pL, ch, f)$ , where  $i$  represents a random universal unique identifier for the node,  $a$  refers to the activity label,  $p$  represents the parent node or predecessor (referencing the directly-follows relation),  $pL$  refers to the list of all parent nodes for this node (starting from the root node and up until the parent node. This same list could be represented as  $pL = \langle p_0, p_1, p_2, \dots, p_m \rangle$  where  $p_0$  represents the root node and  $p_m$  the last parent node),  $ch$  represents a ordered pair storing all the children nodes of this node (this dictionary can be represented as  $ch = \{a : n\}$  where  $a$  refers to the activity of the children node and  $n$  to the node itself; each child node would be an entry in the ordered

pair), and  $f$  represents the frequency of this node (number of times this sequence of events has been seen).

Cases are represented with  $c = (ci, n, fl)$ , where  $ci$  refers to the case identifier,  $n$  represents the last node seen for that case and  $fl$  a Boolean flag to note if the case is still active (open) or inactive (closed).

#### IV. PrefixCDD: AN ONLINE CDD TECHNIQUE

Figure 2 shows a high level representation of the framework's components. Following subsections will give a detailed explanation on each step as referred in the Figure 2. Figure 3 details the *PrefixCDD* algorithm.

##### A. Pre-processing the Event Stream

The first step involves extracting the data from  $S$  and store it in our efficient data structure: prefix-trees. The algorithm will extract each event  $e$  and store its information in a tree until the initial window of observation  $W$  is full. Total window size is defined as:  $W = k \times j$  where  $k$  is the number of trees,  $j$  is the number of events per tree.

Once  $W$  is filled with trees, we need to convert such discrete data into a continuous data set in order to apply a drift detection technique. This transformation technique must take in consideration both the topological structure of the trees, as well as the nodal attributes stored.

Because of that, we use the framework in [15], which is a mathematical framework that helps for performing statistical analysis over populations of tree-structured objects. According to the framework, PCA-based methods can be used to analyze variations w.r.t. a reference point. This led to the introduction of a metric  $\delta$  on tree space. Such metric consists of two parts: the integer part  $d_I$ , which captures the topological structure of the tree, and the fractional part  $f_\delta$ , which captures characteristics of the nodal attributes.

Figure 4 shows how we compute the metric over two trees. Nodes linked by dotted-lines are identical from a topological perspective and activity  $a$ , while those highlighted in red are present in one tree but not the other. The metric would then count the symmetric set difference between both (red nodes), plus and the sum of square differences of the identical nodes' aggregated frequencies, and the sum of squares of the symmetric set frequencies.

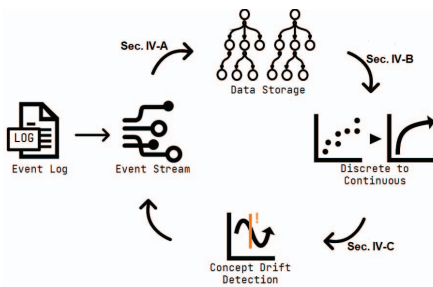


Fig. 2. *PrefixCDD* components and the corresponding sections detailing them.

##### Algorithm 1: PrefixCDD Algorithm

```

Input: EventLog: input to generate event stream,
k: window maximum number of trees,
j: number of events per tree
1 log ← readEventLog(EventLog); /* using PM4PY */
2 S ← convertToStream(log)
3 Wmax ← k × j
4 initialize W, Tree, ADWIN
5 WinSize = Wmax
6 foreach e in S do
7   Tree.insertEvent(e)
8   if len(PrefixTreeList) = WinSize then /* W is full */
9     foreach W partition do
10      W0, W1 ← split(W)
11      ADWIN.append(TreeDistance(W0, W1))
12      if ADWIN.isChange() then /* Drift Detected */
13        W.drop(W0)
14        WinSize = size(W1)
15      else /* No drift detected */
16        W.PrefixTreeList ← W.dropLast()
17      end
18   end
19 end

```

Fig. 3. A Pseudocode of *PrefixCDD* Algorithm.

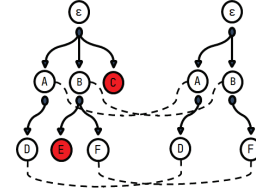


Fig. 4. Tree distance metric representation. Linked nodes are identical. Red nodes have no counterparts over the other tree.

This metric  $\delta(s, t)$  is used as the distance between two trees  $s$  and  $t$ . If the returned value is large, the two trees are said to be different, either from the structure or the frequencies of the nodes, or both. A value close to zero means both trees are almost similar from both perspectives. Storing the values that are computed over a list of contiguous trees will generate a continuous representation of the evolution of the trees, i.e. the evolution of the process.

##### B. Concept Drift Detection

As previously mentioned, most unsupervised change detection techniques rely on comparing the data distribution between two windows of observation. In *PrefixCDD*, we utilize ADWIN [16] for detecting the concept drifts. ADWIN uses an adaptive window  $W$  and checks for possible cuts of  $W$  into subwindows  $W_0$  (reference window) and  $W_1$  (test window) according to the difference in their means. When ADWIN finds a partitioning with  $\mu(W_0, W_1) > \epsilon_{cut}$  for a predefined threshold  $\epsilon_{cut}$ , it shrinks the main window and signals a drift. We maintain an ADWIN object to store the distance metric values that are computed over every partition of our window  $W$  and signal a drift once this value exceeds a computed threshold, generated by [16]. Once a drift is signaled, data from the  $W_0$  is discarded and  $W_1$  becomes the

new reference population. Furthermore, the size of  $W$  shrinks to focus on the new changes in the data and make the concept drift detector more sensitive for the subsequent changes. If no drift is detected,  $W$  size will continue to increase until the window is filled, which means that the distribution has not changed. Figure 5 shows a representation of the adaptive window logic implemented.

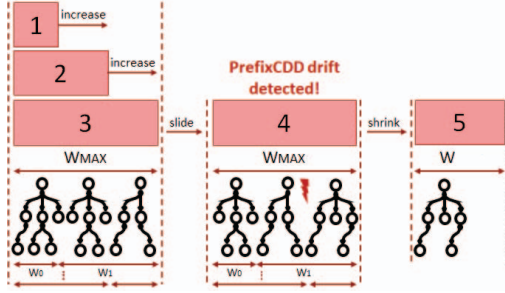


Fig. 5. Adaptive window:  $W$  increases while underlying distribution does not change. Once it reaches maximum size, CDD is performed over every partition; if no drift detected,  $W$  slides and continues; otherwise, the framework forgets reference window and shrinks the window of observation.

The maximum window ( $W_{MAX}$ ) in Step 4 from Figure 5 generates three prefix-trees out of the event stream (cf. Sec. IV-A). For one of the window partitions into  $W_0$  and  $W_1$ , two aggregated trees are generated. The distance between these is computed using the tree distance metric ( $\delta$ ) defined in Sec. IV-A. This value is passed as input to the ADWIN object, which signals a drift (cf. Sec. IV-B). The is repeated for all  $W_{MAX}$  partitions over all events of Stream ( $S$ ).

## V. EXPERIMENTAL EVALUATION

This section presents the utilizes datasets, the competitors, the evaluation metrics, the results and the analysis. A Python implementation of PrefixCDD is available here<sup>1</sup>.

### A. Datasets

We evaluated our framework over a set of synthetic and real-world datasets. The six synthetic event logs are taken from the 72 logs used in [12]. In the synthetic logs, the base process model is altered by applying different change patterns defined in [17] every 10% of the log, generating therefore nine drifts. We used logs of 2500 traces with six *composite change patterns* (“IOR”, “IRO”, “OIR”, “ORI”, “RIO”, “ROI”).

Three real-world datasets were used for this experimental evaluation. Different editions of the Business Process Intelligence Challenge (BPIC) were used to generate the three datasets. I) The Dutch Academic Hospital (2011<sup>2</sup>) and the Dutch Financial Institute (2012<sup>3</sup>) were intertwined by adjusting the event dates from the smaller dataset (2011) to subsist within the larger dataset (2012). II) The five event logs from a Dutch building permit application processes

(2015<sup>4</sup>) were sequentially merged to generate 4 drifts (one after every merge). III) Finally, the travel permits submitted by the TU/e staff (2020<sup>5</sup>) contains a *gradual drift* when the process was extended to multiple departments. Table II shows characteristics of each event log.

| Real Dataset        | Log Size (events) | # of Drifts | Type of Drift |
|---------------------|-------------------|-------------|---------------|
| BPI 2011 + BPI 2012 | 412,491           | 2           | Sudden        |
| BPI 2015            | 262,628           | 4           | Sudden        |
| BPI 2020            | 72,151            | 1           | Gradual       |

TABLE II  
BUSINESS PROCESS CHALLENGE DATASETS CHARACTERISTICS

### B. Experimental Setup

We first evaluate the impact of the window size parameters (max number of trees and number of events per tree) on the accuracy of our results. Using the six synthetic event logs, we assess the accuracy of our method using the F1-score. To compute the F1, a lag period  $l$  surrounding a detected or actual concept drift will be used. It is computed as  $l = 2 \times j$ , where  $j$  is the number of events per tree. A detected concept drift is classified as True Positive (TP) if the time  $t$  of identification is within  $t \pm l$  from the actual drift. A drift is classified as False Positive (FP) if there is no actual change within  $t \pm l$  from the identified drift. A False Negative (FN) is signaled if there is an actual drift at time  $t$ , but no drift was identified by the method within  $t \pm l$ . We also include the *mean delay*, measured as the number of events between the actual position of the drift and the end of the detection window.

Our first experiment is used for tuning the hyper-parameters by measuring the impact of the different values on the model’s performance. Once the best values are identified, the accuracy of our method is compared against three baseline methods; namely, ProDrift [12], Robust [14], and DOA [10]. Finally, we report the running times as another performance metric.

### C. Parameter Tuning

For better selection of the hyper-parameters’ values, we designed two experiments where we used different number of trees in each window of observations and analyzed the impact of the window size on the F1 (Figure 6) and the detection delay (Figure 7). In both experiments, we fix the number of events per tree to 500 events based on the initial results that we got during the development phase. Finally, We carried out an additional experiment to confirm that the selection of 500 events per tree is the best choice. Intuitively, increasing the window size would result in a better representation of the stream using aggregated trees that are generated for each partition. However, when the window size is very large, the aggregated trees are too similar and ADWIN will fail to identify concept drifts. That will increase the FNs ratio. With small window sizes the trees do not store enough events to create an accurate representation of the stream. Therefore, the

<sup>1</sup>PrefixCDD: <https://github.com/jhuete/PrefixTreeCDD>

<sup>2</sup>BPI 2011: <https://www.win.tue.nl/bpi/2011/challenge.html>

<sup>3</sup>BPI 2012: <https://www.win.tue.nl/bpi/2012/challenge.html>

<sup>4</sup>BPI 2015: <https://www.win.tue.nl/bpi/2015/challenge.html>

<sup>5</sup>BPI 2020: <https://icpmconference.org/2020/bpi-challenge/>

trees will be too different from each other and generate a high number of FPs. For the synthetic event logs, our experiments show that the best number of trees (that improves F1) is 12 trees. Figure 6 shows the results for different number of trees over the six synthetic logs.

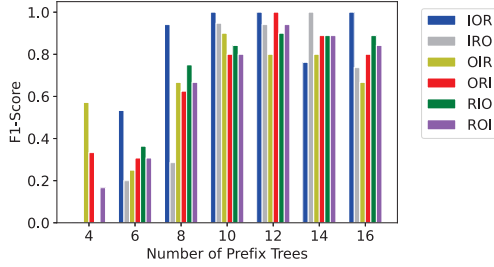


Fig. 6. F1 for different number of trees per  $W$  over the six synthetic logs.

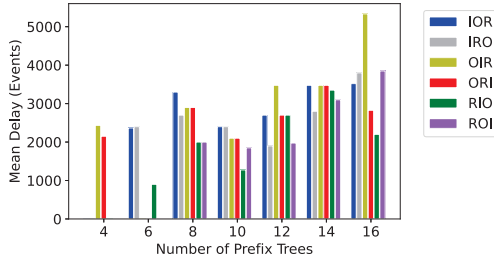


Fig. 7. Mean delay (events) for different maximum number of trees for Window  $W$  over the six synthetic logs.

For the second experiment, we can observe an increase in the detection delay when the window size is large (Figure 7). This can be explained as the higher number of trees requires higher number of events to generate distinct trees and identify drifts. On the other hand, with a smaller number of trees, the delay decreases and the reported FPs increase. We can conclude that using 12 prefix trees is the best choice.

Finally, we study the impact of the number of events per tree over the F1. For this experiment, we select 12 trees as the size of our window of observation. The results show a clear lead on trees storing 500 events than any of the other sizes for the six logs. While smaller sizes identify a high amount of FPs, bigger window sizes are not able to identify all the known-drifts (Figure 8). Based on these experiments, we use 12 prefix trees and 500 events per tree in the following experiments.

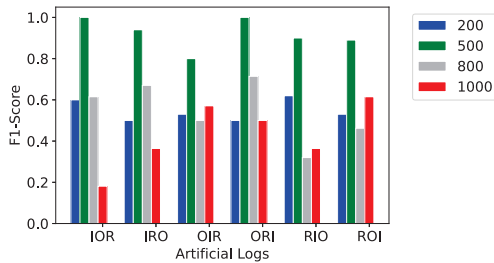
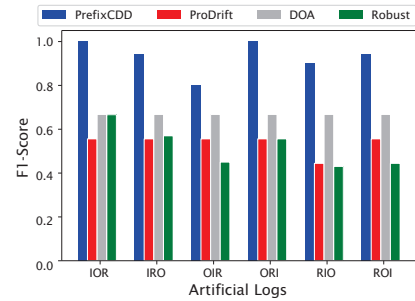


Fig. 8. F1 for different number of events per tree over the six synthetic logs.

#### D. Comparison with the Baseline Methods on Synthetic Logs

Next, we compare PrefixCDD against three baseline methods: ProDrift [12], Robust [14], and DOA [10]. These three methods have shown to be mature in identifying sudden, gradual (ProDrift and Robust) and incremental (DOA) drifts, while utilizing different approaches for analyzing the event stream (feature selection for ProDrift and Robust and process discovery for DOA). We use the six synthetic event logs.

When running the baseline methods, we use the default parameters that are suggested by the authors. For ProDrift, it was recommended to use *Medium*, *High*, and *Very High* for the drift sensitivity with artificial datasets. Therefore, the three parameters are used for each event log and the best resulting ones in terms of *F1* and *mean delay* are selected. For Robust, the recommended parameters are 1500 events for the window size and 2 number of consecutive tests. Figure 9 shows that our technique outperforms the three competitors in terms of F1. Both ProDrift and Robust are unable to detect the known-drifts on the second half of the six event logs. Furthermore, DOA did not show a difference in the results, regardless of which event log is used. It appears that because the nine drifts are located at the exact same position within the event log, the process models generated by the technique do not generate a significant disparity on the number of non-conforming traces. The data table in Figure 9 shows the results in terms of mean delay (events) for our framework and ProDrift. Different to the competitors, PrefixCDD identifies most of the known-drifts. Both Robust and DOA are offline techniques and require to process all events from the event stream before detecting any drift. Therefore, their detection delay is considered equal to the size of the log.



|           |        |       |       |       |         |        |
|-----------|--------|-------|-------|-------|---------|--------|
| PrefixCDD | 2700   | 1900  | 3475  | 2700  | 2700    | 1975   |
| ProDrift  | 1405.2 | 666.2 | 632.6 | 893.2 | 1708.75 | 1311.6 |

Mean Delay (Events)

Fig. 9. Comparing the evaluated methods in terms of F1 and mean delay.

#### E. Evaluation on Real-World Event Logs From Table II

**First:** the five event logs provided in BPI 2020 dataset are used to evaluate our framework against the three competitors. This dataset contains a *gradual drift* (cf. Table II). For ProDrift, the sensitivity of detection was set to *Low* and *Very Low*. According to the provided documentation, such parameter setting performs better over real-world datasets. For Robust, we used the parameters that are recommended in [14].

DOA is not evaluated over real-world datasets due to its low performance and the high time complexity.

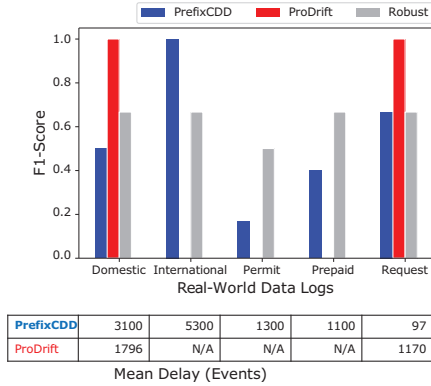


Fig. 10. Comparing PrefixCDD with ProDrift and Robust in terms of F1 and mean delay per event log.

Figure 10 shows the results of this experiment. Our technique is able to beat ProDrift in terms of F1 in three of the event logs, and only in one event log against Robust. Though our technique is able to identify all known-drifts ( $Recall = 1$ ) over the five event logs, it also identified a high number of FPs in all logs except one (International Declarations). ProDrift was unable to identify the known-drift in three of the event logs, while Robust identified all known-drifts, but due to its methodology of performing a forward and backward analysis, it identified each drift twice at different time points; therefore affecting its results for the F1. The table in the bottom of Figure 10 shows results in terms of mean delay of detection. Our technique has a higher delay than ProDrift in one of the event logs (Domestic Declarations), but outperforms it in another (Request for Payment). Furthermore, since ProDrift is unable to detect the known-drifts in three event logs, we can conclude that our technique beats it over delay of detection as well. Since Robust is an offline drift detection technique, its delay of detection would involve the complete event log. PrefixCDD is shown to perform well in detecting the gradual drifts when the size of the traces is big enough so the prefix trees can store a significant number of traces. For those event logs which are smaller in the number of events, the trees are unable to capture enough behavior over gradual drifts.

**Second:** we conducted experiments over the BPI 2015 and the combined BPI 2011+BPI 2012 datasets. These are bigger datasets with a higher number of sudden drifts. Our technique was able to identify all known-drifts ( $Recall = 1$ ) for both datasets (2015 and 2011+2012). It also identified a high amount of FPs, which affected the F1. Our technique was able to process each dataset in less than 20 minutes. We have terminated both ProDrift and Robust scripts after more than three hours of execution with no results for BPI 2015. ProDrift was able to identify one of the two known drifts for BPI 2011 + BPI 2012 after an hour of execution. Table III shows that our algorithm has better time complexity than ProDrift and

Robust for big event logs with sudden drifts.

| maxWindowSize | BPI 2015 | BPI 2011+ BPI 2012 | BPI 2020 |
|---------------|----------|--------------------|----------|
| 6 trees       | 4.158    | 4.65               | 0.183    |
| 8 trees       | 6.64     | 6.41               | 0.216    |
| 10 trees      | 9.4      | 8.63               | 0.233    |
| 12 trees      | 13.88    | 11.43              | 0.225    |
| 14 trees      | 14.89    | 13.06              | 0.283    |
| 16 trees      | 14.41    | 19.35              | 0.25     |
| ProDrift [12] | ~60      | >>180              | >>180    |

TABLE III  
RUNNING TIMES IN MINUTES FOR BPI 2015, BPI 2020, AND THE COMBINED BPI 2011 + BPI 2012 REAL EVENT LOGS USING PREFIXCDD.

## VI. CONCLUSION AND OUTLOOK

We presented an unsupervised online concept drift detection technique for detecting sudden drifts from event streams. The framework stores the event stream data as a set of prefix-trees and applies an accurate distance metric to compute a concept drift score. Moreover, PrefixCDD is able to identify gradual drifts. An evaluation over six synthetic and three real-world datasets show that PrefixCDD outperforms three baseline methods (including two offline approaches) in terms of F1 and detection speed. PrefixCDD runs considerably faster.

## REFERENCES

- [1] M. Hassani, S. Siccha, F. Richter, and T. Seidl, "Efficient process discovery from event streams using sequential pattern mining," in *IEEE SSCI*, 2015.
- [2] M. Hassani, S. J. van Zelst, and W. M. P. van der Aalst, "On the application of sequential pattern mining primitives to process discovery: Overview, outlook and opportunity identification," *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.*, vol. 9, 2019.
- [3] S. J. van Zelst, A. Bolt, M. Hassani, B. F. van Dongen, and W. M. P. van der Aalst, "Online conformance checking: relating event streams to process models using prefix-alignments," *Int. JDSA*, vol. 8, 2019.
- [4] R. Zaman, M. Hassani, and B. F. V. Dongen, "Prefix imputation of orphan events in event stream processing," *Fr. in Big Data*, vol. 4, 2021.
- [5] S. Pauwels and T. Calders, "Incremental predictive process monitoring: The next activity case," in *Business Process Management*, 2021.
- [6] J. C. B. et.al., "Change point detection and dealing with gradual and multi-order dynamics in process mining," in *BIR*, 2015.
- [7] C. C. Aggarwal, Ed., *Data Streams - Models and Algorithms*, ser. Advances in Database Systems, 2007, vol. 31.
- [8] A. A. Qahtan, B. Alharbi, S. Wang, and X. Zhang, *A PCA-Based Change Detection Framework for Multidimensional Data Streams: Change Detection in Multidimensional Data Streams*. ACM SIGKDD, 2015.
- [9] M. Hassani, "Concept drift detection of event streams using an adaptive window," *ECMS*, 2019.
- [10] L. Zellner, F. Richter, J. Sontheim, A. Maldonado, and T. Seidl, "Concept drift detection on streaming data with dynamic outlier aggregation," in *ICPM Workshops*, vol. 406, 2020.
- [11] A. O. et.al., "Detecting drift from event streams of unpredictable business processes," in *ER*, vol. 9974, 2016.
- [12] A. Maaradji, M. Dumas, M. L. Rosa, and A. Ostovar, "Detecting sudden and gradual drifts in business processes from execution traces," *IEEE TKDE*, vol. 29, 2017.
- [13] B. Hompes, J. C. A. M. Buijs, W. M. P. van der Aalst, P. M. Dixit, and H. Buurman, "Detecting change in processes using comparative trace clustering," *SIMPDA*, 2015.
- [14] Y. Lu, Q. Chen, and S. Poon, "A robust and accurate approach to detect process drifts from event streams," in *BPM*, 2021.
- [15] H. Wang and J. S. Marron, "Object oriented data analysis: Sets of trees," *The Annals of Statistics*, vol. 35, 2007.
- [16] A. Bifet and R. Gavaldá, "Learning from time-changing data with adaptive windowing," in *SIAM*, 2007.
- [17] B. Weber, S. Rinderle-Ma, and M. Reichert, "Change patterns and change support features - enhancing flexibility in process-aware information systems," *DKE*, vol. 66, 2008.